

Overview of Convolutional neural network

Subject: Convolutional neural network

$$a^{(l)} = \sigma(W^{(l)}a^{(l-1)} + b^{(l)})$$

1.

Pooling layers Convolutional networks may include local and/or global pooling layers along with traditional convolutional layers. Pooling layers reduce the dimensions of data by combining the outputs of neuron clusters at one layer into a single neuron in the next layer. Local pooling combines small clusters, tiling sizes such as 2×2 are commonly used. Global pooling acts on all the neurons of the feature map. There are two common types of pooling in popular use: max and average. Max pooling uses the maximum value of each local cluster of neurons in the feature map, while average pooling takes the average value.

2.

3D volumes of neurons. The layers of a CNN have neurons arranged in 3 dimensions: width, height and depth. Each neuron inside a convolutional layer is connected to only a small region of the layer before it, called a receptive field. Distinct types of layers, both locally and completely connected, are stacked to form a CNN architecture. Local connectivity: following the concept of receptive fields, CNNs exploit spatial locality by enforcing a local connectivity pattern between neurons of adjacent layers. The architecture thus ensures that the learned "filters" produce the strongest response to a spatially local input pattern. Stacking many such layers leads to nonlinear filters that become increasingly global (i.e. responsive to a larger region of pixel space) so that the network first creates representations of small parts of the input, then from them assembles representations of larger areas. Shared weights: In CNNs, each filter is replicated across the entire visual field. These replicated units share the same parameterization (weight vector and bias) and form a feature map. This means that all the neurons in a given convolutional layer respond to the same feature within their specific response field. Replicating units in this way allows for the resulting activation map to be equivariant under shifts of the locations of input features in the visual field, i.e. they grant translational equivariance—given that the layer has a stride of one. Pooling: In a CNN's pooling layers, feature maps are divided into rectangular sub-regions, and the features in each rectangle are independently down-sampled to a single value, commonly by taking their average or maximum value. In addition to reducing the sizes of feature maps, the pooling operation grants a degree of local translational invariance to the features contained therein, allowing the CNN to be more robust to variations in their positions. Together, these properties allow CNNs to achieve better generalization on vision problems. Weight sharing dramatically reduces the number of free parameters learned, thus lowering the memory requirements for running the network and allowing the training of larger, more powerful networks.

3.

Time series forecasting Recurrent neural networks are generally considered the best neural network architectures for time series forecasting (and sequence modeling in general), but recent studies show that convolutional networks can perform comparably or even better. Dilated convolutions might enable one-dimensional convolutional neural networks to effectively learn time series dependences. Convolutions can be implemented more efficiently than RNN-based solutions, and they do not suffer from vanishing (or exploding) gradients. Convolutional networks can provide an improved forecasting performance when there are multiple similar time series to learn from. CNNs can also be applied to further tasks in time series analysis (e.g., time series classification or quantile forecasting).

4.

GPU implementations Although CNNs were invented in the 1980s, their breakthrough in the 2000s required fast implementations on graphics processing units (GPUs). In 2004, it was shown by K. S. Oh and K. Jung that standard neural networks can be greatly accelerated on GPUs. Their implementation was 20 times faster than an equivalent implementation on CPU. In 2005, another paper also emphasised the value of GPGPU for machine learning. The first GPU-implementation of a CNN was described in 2006 by K. Chellapilla et al. Their implementation was 4 times faster than an equivalent implementation on CPU. In the same period, GPUs were also used for unsupervised training of deep belief networks. In 2010, Dan Ciresan et al. at IDSIA trained deep feedforward networks on GPUs. In 2011, they extended this to CNNs, accelerating by 60 compared to training CPU. In 2011, the network won an image recognition contest where they achieved superhuman performance for the first time. Then they won more competitions and achieved state of the art on several benchmarks. Subsequently, AlexNet, a similar GPU-based CNN by Alex Krizhevsky et al. won the ImageNet Large Scale Visual Recognition Challenge 2012. It was an early catalytic event for the AI boom. Compared to the training of CNNs using GPUs, not much attention was given to CPU. (Viebke et al 2019) parallelizes CNN by thread- and SIMD-level parallelism that is available on the Intel Xeon Phi.

5.

ReLU layer ReLU is the abbreviation of rectified linear unit. It was proposed by Alston Householder in 1941, and used in CNN by Kunihiko Fukushima in 1969. ReLU applies the non-saturating activation function $f(x) = \max(0, x)$. It effectively removes negative values from an activation map by setting them to zero. It introduces nonlinearity to the decision function and in the overall network without affecting the receptive fields of the convolution layers. In 2011, Xavier Glorot, Antoine Bordes and Yoshua Bengio found that ReLU enables better training of deeper networks, compared to widely used activation functions prior to 2011. Other functions can also be used to increase nonlinearity, for example the saturating hyperbolic tangent $f(x) = \tanh(x)$, $f(x) = |\tanh(x)|$, and the sigmoid function $\sigma(x) = \frac{1}{1 + e^{-x}}$. ReLU is often preferred to other functions because it trains the neural network several times faster without a significant penalty to generalization accuracy.

6.

Stochastic pooling A major drawback to dropout is that it does not have the same benefits for convolutional layers, where the neurons are not fully connected. Even before dropout, in 2013 a technique called stochastic pooling, the conventional deterministic pooling operations were replaced with a stochastic procedure, where the activation within each pooling region is picked randomly

according to a multinomial distribution, given by the activities within the pooling region. This approach is free of hyperparameters and can be combined with other regularization approaches, such as dropout and data augmentation. An alternate view of stochastic pooling is that it is equivalent to standard max pooling but with many copies of an input image, each having small local deformations. This is similar to explicit elastic deformations of the input images, which delivers excellent performance on the MNIST data set. Using stochastic pooling in a multilayer model gives an exponential number of deformations since the selections in higher layers are independent of those below.